

Modeling Smart Inverters in Distribution Optimal Power Flow

Adedoyin Inaolaji

Assistant Professor

University at Buffalo

The State University of New York



 [Scan or click](#)

Get ready in four steps

1 Install Julia 1.10 or newer: juliaup, the Microsoft Store on Windows, or julialang.org/downloads.

2 Get the code

```
git clone https://github.com/epsrlab-ub/SmartInverter-3P-DOPF-POWERUP.jl
cd SmartInverter-3P-DOPF-POWERUP.jl
```

3 Instantiate the environment

```
julia --project=examples/three_phase -e "using Pkg; Pkg.instantiate()"
```

4 Solvers: Ipopt (open source) runs the Heaviside route on its own; Gurobi, free for academic use, runs the two MILP routes.

Prerequisites — scan it



[epsrlab-ub.github.io/SmartInverter-3P-DOPF-POWERUP.jl
/dev/tutorial_voltvar/#Prerequisites](https://epsrlab-ub.github.io/SmartInverter-3P-DOPF-POWERUP.jl/dev/tutorial_voltvar/#Prerequisites)

*Install notes, the package list and the solver options,
including what to do without a MILP licence.*

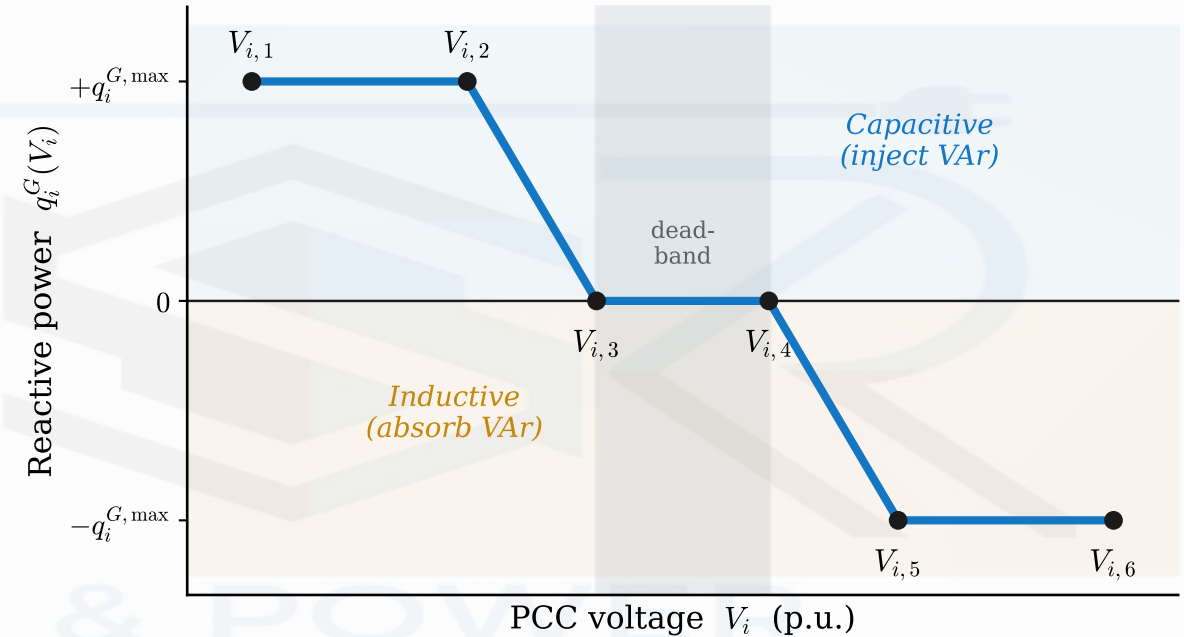
Why embed smart-inverter droop curves into OPF?

The setting

- Smart inverters (SIs) on PV systems provide **voltage support** via reactive power, governed by a **Volt-VAr (Q-V) droop** law.
- IEEE 1547-2018 requires **Volt-VAr capability**.
- An **advanced distribution management system (ADMS)** can coordinate many SIs through a **distribution optimal power flow (DOPF)**, but only if the droop law lives *inside* the optimization.

Why embed it at all?

- SIs don't accept a reactive-power setpoint: they follow their Q-V curve. An OPF that ignores it returns a **reactive power setpoint that may not be feasible at the local inverter controller level**.
- Embedding the curve forces every dispatch point to **lie on the droop**, so the OPF solution is physically realizable and IEEE-1547 compliant.

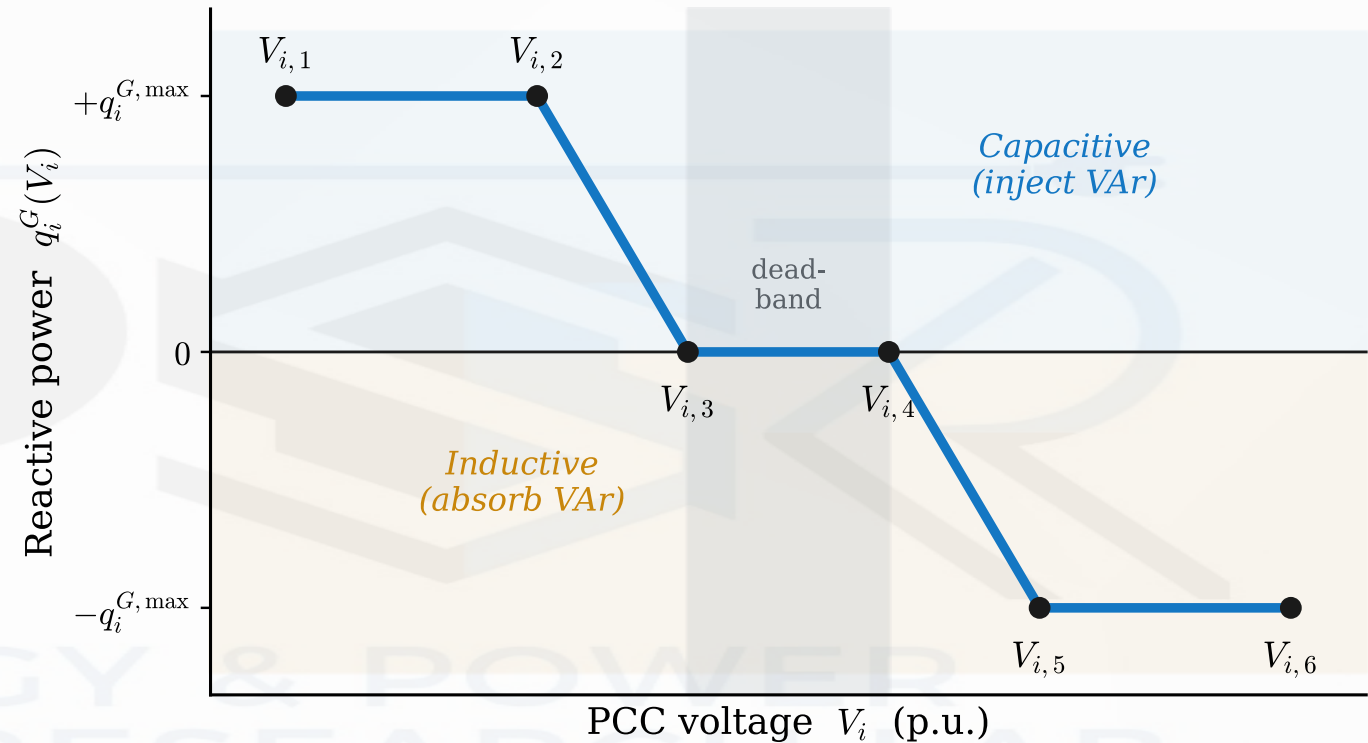


The Volt-VAr control law: a 5-segment piecewise-linear function

Six breakpoints, three regions

- Flat saturation at full inject / absorb.
- Two sloped segments and a central **dead-band**.

$$q(V_i) = \begin{cases} +q_i^{G,\max}, & V_{i,1} \leq V_i \leq V_{i,2} \\ -\frac{q_i^{G,\max}}{V_{i,3} - V_{i,2}} V_i + \frac{q_i^{G,\max} V_{i,3}}{V_{i,3} - V_{i,2}}, & V_{i,2} \leq V_i \leq V_{i,3} \\ 0, & V_{i,3} \leq V_i \leq V_{i,4} \\ -\frac{q_i^{G,\max}}{V_{i,5} - V_{i,4}} V_i + \frac{q_i^{G,\max} V_{i,4}}{V_{i,5} - V_{i,4}}, & V_{i,4} \leq V_i \leq V_{i,5} \\ -q_i^{G,\max}, & V_{i,5} \leq V_i \leq V_{i,6} \end{cases}$$



The same law, per phase

$$q_i^{G,\psi} = q(V_i^\psi), \quad \forall \psi \in \Psi_i \subseteq \{a, b, c\}$$

five segments and six breakpoints per phase; the two sloped segments have slopes

Continuous and piecewise-linear, but non-smooth at the breakpoints.

A solver optimizes over *algebraic* constraints. The droop is defined by if-else logic, leading to two obstacles.

1 — Conditional logic

The segment that is active depends on the phase voltage:

if $V_{i,b}^\psi \leq V_i^\psi \leq V_{i,b+1}^\psi$ then $q_i^{G,\psi} = a_b V_i^\psi + c_b$

Branching logic is not an algebraic constraint.

2 — Non-differentiability

The slope jumps at every kink:

$\frac{d q_i^{G,\psi}}{d V_i^\psi}$ is discontinuous at every breakpoint $V_{i,b}^\psi$

and it jumps on each phase independently.

Breaks Newton / interior-point solvers.

Two ways out:

- (A) introduce integer variables to encode the logic exactly: MIP.
- (B) write the logic in closed algebraic form (step functions): NLP.



↗ Scan or Click



↗ Scan or Click

Scan it and run it: the droop as a Julia if–else, handed to a solver. The model does not even build.

Three ways to embed the Q-V droop

Big-M

○ MILP

- One binary per segment, per phase, activates that segment's affine law and voltage range.

Lambda (SOS2)

○ MILP

- Write (V_i, q_i^G) as a convex combination of the breakpoints, per phase; SOS2 lets only two adjacent weights be nonzero.

Heaviside

○ NLP

- Build segment masks from unit steps $H(V_i - V_{i,b})$. *One sum per phase, no integers, but non-convex.*

One binary per segment activates that segment's affine law and voltage range, **once for every phase the inverter is connected to**.

Per phase $\psi \in \Psi_i$

One binary picks the active segment, on each phase:

$$\delta_{i,m}^\psi \in \{0,1\}, \quad \sum_{m=1}^5 \delta_{i,m}^\psi = 1$$

the chosen segment must contain the sensed voltage:

$$V_{i,m}^\psi - M(1 - \delta_{i,m}^\psi) \leq V_i^\psi \leq V_{i,m+1}^\psi + M(1 - \delta_{i,m}^\psi)$$

and it selects the segment law:

$$q_i^{G,\psi} = \sum_{m=1}^5 \delta_{i,m}^\psi f_m^\psi(V_i^\psi)$$

the products $\delta \cdot V$ are bilinear, so replace them, per phase:

$$W_{i,m}^\psi := \delta_{i,m}^\psi V_i^\psi, \quad V_{i,m}^\psi \delta_{i,m}^\psi \leq W_{i,m}^\psi \leq V_{i,m+1}^\psi \delta_{i,m}^\psi$$

Trade-offs

Pros

- + Exact, solved by any MILP solver.
- + The reformulation–linearization is systematic and mechanical.

Cons

- Five binaries per phase, per inverter, per time step.
- Big-M tuning: a loose M weakens the LP relaxation.

Every point of the curve is a blend of two **adjacent** breakpoints. The weights are the unknowns, and each phase carries its own set.

Per phase $\psi \in \Psi_i$

The operating point is a weighted average of the breakpoints:

$$V_i^\psi = \sum_{b=1}^6 \lambda_{i,b}^\psi V_{i,b}^\psi, \quad q_i^{G,\psi} = \sum_{b=1}^6 \lambda_{i,b}^\psi \bar{q}_{i,b}^\psi$$

the weights are a convex combination, and one segment is on:

$$\sum_{b=1}^6 \lambda_{i,b}^\psi = 1, \quad \lambda_{i,b}^\psi \geq 0, \quad \sum_{b=1}^5 z_{i,b}^\psi = 1, \quad z_{i,b}^\psi \in \{0,1\}$$

adjacency (SOS2), only two neighbouring weights may be nonzero:

$$\lambda_{i,1}^\psi \leq z_{i,1}^\psi, \quad \lambda_{i,b}^\psi \leq z_{i,b-1}^\psi + z_{i,b}^\psi \quad (b = 2, \dots, 5), \quad \lambda_{i,6}^\psi \leq z_{i,5}^\psi$$

$\bar{q}_{i,b}^\psi$ is the droop ordinate at breakpoint b , a known constant, not a variable.

Trade-offs

Pros

- + Exact, the dispatch is on the IEEE 1547 curve.
- + No Big-M constant to tune.
- + SOS2 is supported natively by MILP solvers.

Cons

- Six weights and five binaries per phase, per inverter, per time step.

An “if” is an on/off switch. Written that way the droop becomes one algebraic expression per phase, **with no integer variables at all**.

Per phase $\psi \in \Psi_i$

$$H(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases} \Rightarrow H(V_i - V_{i,b}) = \begin{cases} 0, & V_i < V_{i,b} \\ 1, & V_i \geq V_{i,b} \end{cases}$$

Two unit steps, subtracted, make a window that is 1 on one segment:

$$W_{i,b}^\psi := H(V_i^\psi - V_{i,b}^\psi) - H(V_i^\psi - V_{i,b+1}^\psi), \quad b = 1, \dots, 5$$

mask each segment law and add; the windows are disjoint:

$$q_i^{G,\psi} = q_i^{G,max,\psi} W_{i,1}^\psi + \alpha_{i,1}^\psi (V_i^\psi - V_{i,3}^\psi) W_{i,2}^\psi \\ + \alpha_{i,2}^\psi (V_i^\psi - V_{i,4}^\psi) W_{i,4}^\psi - q_i^{G,max,\psi} W_{i,5}^\psi$$

with the slopes anchored at their zero crossings:

$$\alpha_{i,1}^\psi = \frac{-q_i^{G,max,\psi}}{V_{i,3}^\psi - V_{i,2}^\psi}, \quad \alpha_{i,2}^\psi = \frac{-q_i^{G,max,\psi}}{V_{i,5}^\psi - V_{i,4}^\psi}$$

Trade-offs

Pros

- + Exact, no approximation of the curve.
- + Integer-free, so the binary count does not grow with the number of phases.
- + Compact: one algebraic expression per inverter and phase.

Cons

- $H(\cdot)$ is discontinuous, it jumps at every breakpoint, so the derivative is undefined there.
- Gradient-based NLP solvers struggle.
- Non-convex.

At any voltage all windows but one are **zero**, so the sum collapses to the single active law. This is the if-else, in algebra.

Method-by-method trade-offs

Method	Class	Extra variables per inverters per bus	Curve	Solved by	Watch out for
Big-M	MILP	5 binaries δ per SI	exact	branch & bound, any MILP solver	loose $M \Rightarrow$ weak LP relaxation
Lambda / SOS2	MILP	6 weights λ + 5 binaries z per SI	exact	branch & bound, native SOS2 support	binaries grow with SIs $\times$ time steps
Heaviside	NLP	none	exact, but jumps at the breakpoints	smoothed step $\rightarrow$ NLP (e.g., IPOPT)	derivative undefined at every breakpoint

All three encodings represent the **same curve** and return the **same dispatch**.
The choice is which approach is most suitable: **integer MILP** or **NLP**.

The droop is a **self-contained module**. Whatever three-phase DOPF you use, it exposes each inverter's own phase voltage V_i^ψ ; the droop adds the relation that ties the inverter's reactive output to it.

The droop as a plug-in module

Three-phase DOPF
network model + operating limits



exposes V_i^ψ



sets $q_i^{G,\psi}$

Q-V droop module
 $q_i^{G,\psi} = q(V_i^\psi)$, the IEEE 1547 curve

Two three-phase hosts in this tutorial

- **LinDist3Flow**: multiphase linearised branch flow. Linear, solved in one pass.
- **Three-phase IVACOPF**: current-voltage AC-OPF. Near-exact, successively linearised and iterated.

All three encodings are **independent of this choice**: they ask the host for one scalar voltage per inverter terminal, and nothing else.

Same feeder, same fleet, same droop, two hosts: *running both is what separates what the encoding is responsible for from what the network model is.*

The multiphase form of LinDistFlow. Each line carries a **3×3 phase impedance**, so a scalar $rP + xQ$ drop no longer suffices: the phases couple through the drop itself.

The coupled voltage drop

$$v_j^\varphi = v_i^\varphi - \sum_{\psi \in \Psi} \left(\tilde{a}_{\varphi\psi}^R P_{ij}^\psi + \tilde{a}_{\varphi\psi}^X Q_{ij}^\psi \right)$$

with the 120° rotation $\alpha = e^{-j2\pi/3}$ baked into the coefficients:

$$a_{\varphi\psi}^R = 2 \operatorname{Re}(\alpha^{\psi-\varphi} Z_{\varphi\psi}), \quad a_{\varphi\psi}^X = 2 \operatorname{Im}(\alpha^{\psi-\varphi} Z_{\varphi\psi}), \quad \tilde{a} = a / 2V^{\text{nom}}$$

Lossless power balance, the slack $v_0^\varphi = V^{\text{nom}}$, and the band $V^{\min} \leq v_j^\varphi \leq V^{\max}$.

What it assumes, buys, costs

Assumes: the inter-phase voltage ratio fixed at $1\angle\pm 120^\circ$ and the network has no losses.

Buys: a linear model: one pass, no iteration, and by far the cheapest of the two hosts.

Costs: losses are dropped and the power flow is not exact.

Use it for a fast first look, and to warm-start the near-exact host.

For the droop, v_j^φ is a decision variable directly: the module reads it, and nothing else.

Write the network in **rectangular current and voltage coordinates** and the line equations become exactly linear, mutual coupling and all. The only nonlinearity left lives at the buses.

Lines: exact and linear

$$v_n^{re,\varphi} - v_m^{re,\varphi} = \sum_{p \in \Psi} \left(R_{nm}^{\varphi p} I_{nm}^{re,p} - X_{nm}^{\varphi p} I_{nm}^{im,p} \right)$$
$$v_n^{im,\varphi} - v_m^{im,\varphi} = \sum_{p \in \Psi} \left(R_{nm}^{\varphi p} I_{nm}^{im,p} + X_{nm}^{\varphi p} I_{nm}^{re,p} \right)$$

No rotation operator, nothing transposed, and near-balance assumed nowhere. KCL at each bus and phase is linear too.

Buses: the only nonlinearity

$$p_n^{G,\varphi} - p_n^{L,\varphi} = v_n^{re,\varphi} I_n^{re,\varphi} + v_n^{im,\varphi} I_n^{im,\varphi}$$
$$v_n^\varphi = \sqrt{(v_n^{re,\varphi})^2 + (v_n^{im,\varphi})^2}$$

Both are linearised about the previous iterate and the model re-solved, until the residual of the exact power-flow identity clears a tolerance.

Successive linearisation: more passes, near-exact AC accuracy.

1 — Embedding is a correctness requirement

SIs follow their Q-V curve, not a setpoint. Only a droop-aware OPF returns dispatch the inverters will actually deliver.

2 — Two exact families

Binaries (Big-M, Lambda/SOS2 → **MILP**) or nonsmooth algebra (Heaviside → **NLP**). All three encode the same curve, and return the **same dispatch**.

3 — Scale picks the method; breakpoints are the frontier

Binaries multiply with SIs × phases × time steps; the integer-free encoding does not. Letting the OPF **move the breakpoints** adds a bilinear $\lambda \cdot V$, the open challenge.

One curve, many encodings: **choose the one your solver loves**.

Thank You

Questions & discussion

